

Love of Variety Model and Simulation

Tate Mason

March 30, 2026

1 Model

1.1 Consumer's Problem

The consumer's problem is to maximize utility subject to their choice set. The utility function is as follows:

$$U_{ijt} = \beta_i X_{jt} - (\gamma_i \Sigma_j)^2 + a + \xi_j + \epsilon_{ijt} \quad (1)$$

$$\Sigma_j = X_{jt} - \bar{X}_{jt}$$

Where U_{ijt} is the utility of consumer i from product j at time t . X_{jt} is the attribute of product j at time t . γ_i denotes a cost of sameness. Σ_j is the sum of product attributes chosen until time t . Note, γ_i has a standard normal distribution. ξ_j captures utility from unobserved quality. Finally, ϵ_{ijt} is a random error term distributed T1EV.

$$P_{ijt} = \frac{e^{U_{ijt}}}{1 + \sum_k e^{U_{ikt}}} \quad (2)$$

Where the denominator sums over all products k in the choice set at time t , with the outside option.

1.2 Firm's Problem

The firm will maximize profits by choosing how to advertise to the consumer according to their perceptions surrounding the distribution of γ .

$$\pi = (1 + a)p - c * (\kappa * a)^2 \quad (3)$$

Where a is the advertising level and p is the price of the product, c is some baseline cost for the firm, and κ is an advertising specific cost. If the firm chooses not to advertise, they get a profit of $p - c$. If they choose to advertise, they get a profit of $(1 + a)p - c$.

The firm can choose to advertise at different levels, such that $a \in [0, 1]$. This can be conceived of as a loss of revenue in hopes of maintaining a customer. The firm chooses a to maximize profits based on the following function:

$$V(a|\lambda, \Sigma_j, a_{t-1}) = \max_{\pi} \left\{ \pi + \int_{\gamma} V'(a'|\lambda, \Sigma_j, a) d\lambda(\gamma) \right\} \quad (4)$$

Here, $\lambda(\gamma)$ is the firm's belief about the distribution of γ in the consumer population. Σ_j is the sum of product attributes chosen until time t . a_{t-1} is the advertising level at time $t - 1$. The firm chooses a to maximize profits based on its current beliefs and the expected future value of

advertising. Note, the distribution of $\lambda(\gamma) \sim N(\frac{1}{\Sigma_j}, \sigma_\lambda)$. This states they use past purchases to update their mean belief about the distribution of γ , but they have some variance in that belief.

2 Firm Optimization

The firm will choose a to maximize profits π based on the perceived distribution of γ in the consumer population. The firm will also keep track of the sum of product attributes chosen by the consumer, Σ_j , to update their beliefs about γ . The firm will solve the following optimization problem:

$$V(a, \Sigma_j | \lambda(\gamma)) = \max_a \{ \pi(a) + \int_{\gamma} V(a', \Sigma'_j | \lambda(\gamma)) d\lambda(\gamma) \} \quad (5)$$

Where $\pi(a) = (1+a)p - c * (\kappa * a)^2$ is the profit function, $\lambda(\gamma) \sim N(\frac{1}{\Sigma_j}, \sigma_\lambda)$, and $\Sigma'_j = \Sigma_j + X_{jt} \cdot P_{ijt}$ is the updated sum of product attributes chosen by the consumer conditional on choosing product j at time t . The firm's FOC's are as follows:

$$\begin{aligned} \frac{\partial V}{\partial a} &= p + P_{ijt}(1 - P_{ijt}) \int_{\gamma} [V(\Sigma_j + X_{jt} | \lambda'(\gamma)) - V(\Sigma_j | \lambda'(\gamma))] d\lambda(\gamma) = 0 \\ &= p + P_{ijt}(1 - P_{ijt}) [V(\Sigma_j + X_{jt} | \lambda(\Sigma_j + X_{jt})) - V(\Sigma_j | \lambda(\Sigma_j))] = 0 \\ &\Rightarrow p^* = (P_{ijt}(1 - P_{ijt})) \cdot [V(\Sigma_j + X_{jt} | \lambda(\Sigma_j + X_{jt})) - V(\Sigma_j | \lambda(\Sigma_j))] \end{aligned}$$

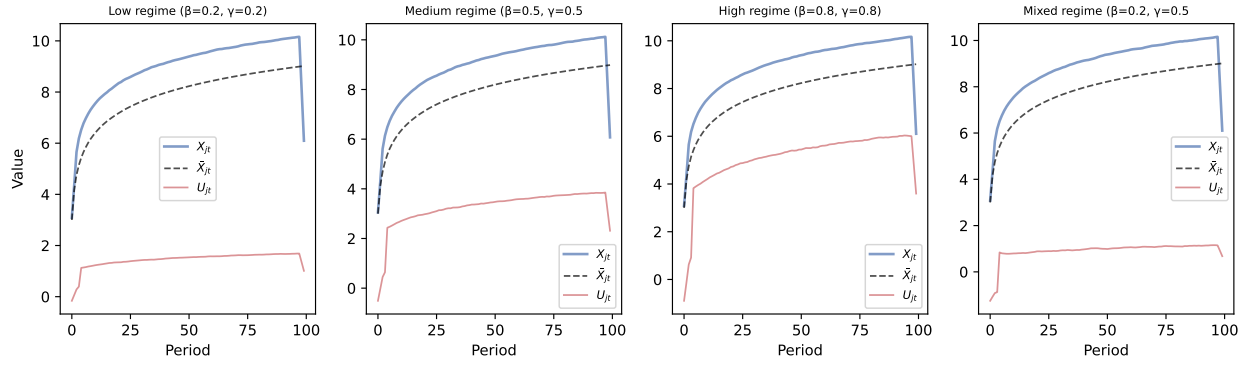
The firm will choose a such that the marginal profit from advertising equals the expected marginal value of advertising in terms of future profits. The firm will update their beliefs about γ based on the consumer's choices and the sum of product attributes chosen, Σ_j .

3 Results

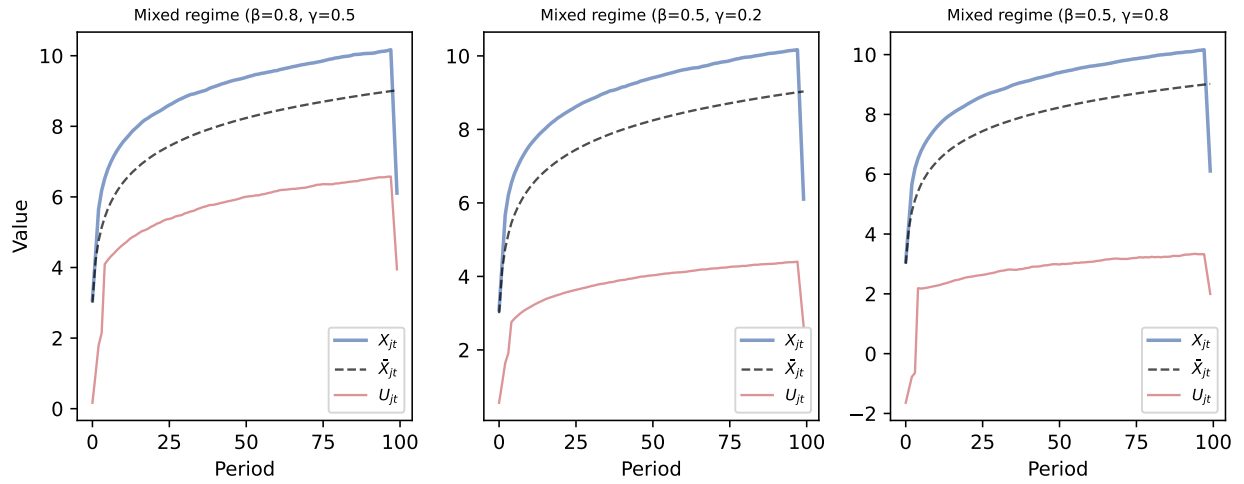
On the consumer side, there are a few things to note. Firstly, as γ grows, the utility becomes noisier, no matter the value of β . Secondly, notice it takes about 10 periods for utility to find some sort of stability. However, as β grows, the time to stabilize grows. For instance, in the low regime, it takes only five or so periods, where as in the high regime, this is closer to 20. These results suggest that β finds a sweet spot in utility and γ fluctuates around it.

In the firm's problem, we look at the firm's advertising decision. In the low regime, they are mostly advertising, with a couple of deviations around period 50 and 75. The rest of the regimes, however, see a lot of variation in the decision to advertise. This is to be expected, since the firm bases its decision based off of its observations of Σ to approximate γ .

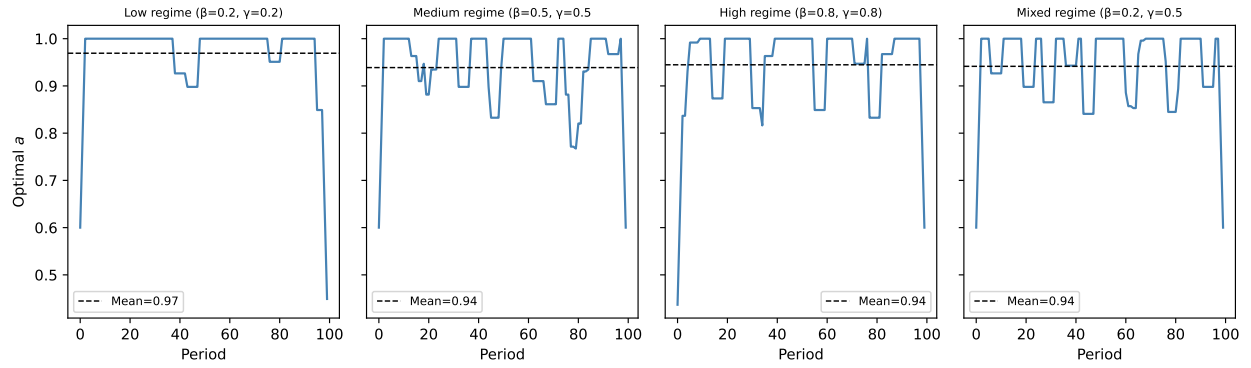
Consumer Choices and Utility by Regime ($\sigma_x = 1.0$)



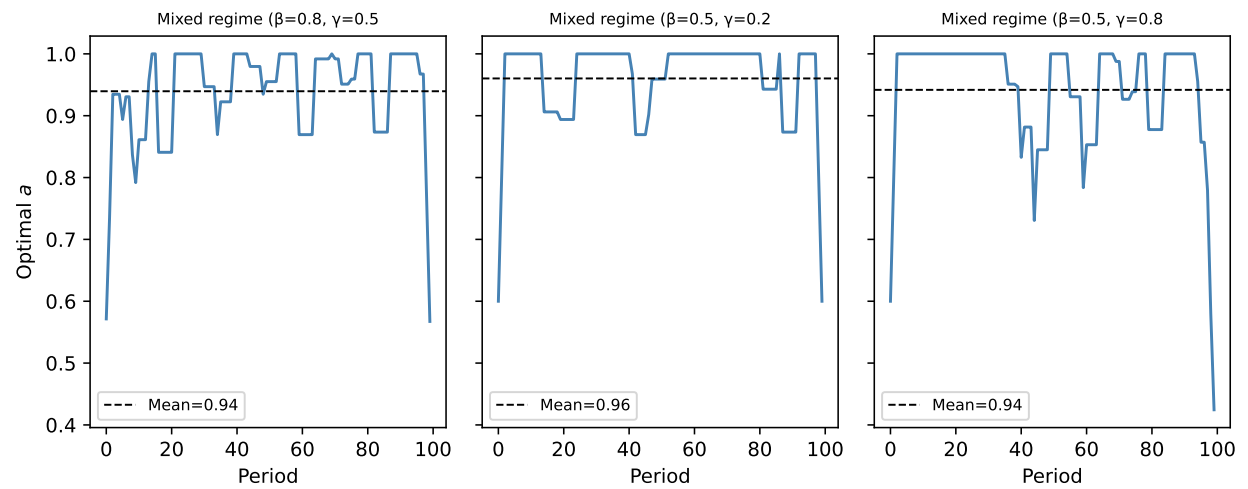
Consumer Choices and Utility by Regime ($\sigma_x = 1.0$)



Firm's Optimal Advertising by Consumer Regime



Firm's Optimal Advertising by Consumer Regime



4 Code

```
import pandas as pd
import os
import numpy as np
import scipy as sp
from scipy.special import logsumexp
import matplotlib.pyplot as plt
import seaborn as sns
from tabulate import tabulate

"""
This file will be used to do initial simulation of the love of variety problem:
-----Consumer Problem:  $U_{ijt} = \beta_i X_{jt} - \gamma_i (\Sigma_{jt}) + a_{ijt} + \epsilon_{ijt}$ 
-----  $s.t. \gamma_i \sim N(0, \sigma_{\gamma})$ ;  $\Sigma_{jt} = \sum_t T_{jt} X_{jt}$ 
-----Firm Problem:  $V(a_t; \Sigma_{jt}, \lambda_t(\gamma_i)) = \{ \pi_t + \int \gamma_i V(a'; \Sigma_{jt} + 1, \lambda_t + 1(\gamma_i)) d\lambda_t(\gamma_i) \}$ 
-----  $s.t. \pi_t = (1+a)p_t - c$ 
-----  $\lambda_t(\gamma_i) \sim N(1/\Sigma_{jt}, \sigma_{\lambda})$ ,  $a_t \in \{0, 1\}$ 
Structure:
1) solve consumer's choice problem
2) compute individual shares of products
3) compute firm's problem, choosing optimal  $a$ 
4) find partial equilibrium
Landscape:
1)  $T = 100$ 
2)  $J = 5$ 
3)  $i = 1$ 
4)  $\gamma = \text{range}(0.2(0.3)0.8)$ ,  $\beta = \text{range}(0.2(0.3)0.8)$ 
Functions:

Libraries Used:
-----numpy: used for numerical operation
-----scipy: used for optimization routines
-----matplotlib, seaborn: used for plotting
-----tabulate: better output text results

"""

#=====#
# Setup and Parameter Definition                                     #
#=====#

output_dir = os.chdir('../Output/')

rng = np.random.default_rng(seed=219)

# Parameterization
```

```

T = 100
sigma_gamma = [0.5, 1.0, 1.5]

regimes = [
    (0.2, 0.2, 'Low_regime_( =0.2, =0.2)', '-.'),
    (0.5, 0.5, 'Medium_regime_( =0.5, =0.5)', ':'),
    (0.8, 0.8, 'High_regime_( =0.8, =0.8)', '--'),
    (0.2, 0.5, 'Mixed_regime_( =0.2, =0.5)', '-'),
    (0.8, 0.5, 'Mixed_regime_( =0.8, =0.5)', '-'),
    (0.5, 0.2, 'Mixed_regime_( =0.5, =0.2)', '-'),
    (0.5, 0.8, 'Mixed_regime_( =0.5, =0.8)', '-'),
    (0.8, 0.2, 'Mixed_regime_( =0.8, =0.2)', '-'),
    (0.2, 0.8, 'Mixed_regime_( =0.2, =0.8)', '-')
]

J = 5
S = 1000
X_bar = rng.standard_normal()
sigma_x = 1.0

# Consumer Choice Problem

def simulate_cons(beta, gamma, sigma_gamma, T, J, X_bar, sigma_x, rng, S=1000):
    x_chosen_all = np.zeros((S, T))
    x_bar_all = np.zeros((S, T))
    X_jt_all = np.zeros((S, T, J))
    V_all = np.zeros((S, T, J))
    ll_all = np.zeros(S)
    probab_all = np.zeros((S, T, J))

    for s in range(S):

        epsilon_ijt = rng.gumbel(0,1, size=J)

        x_chosen = np.zeros(T) # empty vector of choices per period
        X_jt = np.zeros((T,J))
        x_bar = np.zeros(T)
        x_bar[0] = X_bar

        Sigma = np.zeros(T) # empty vector for the sum of characteristics

        V = np.zeros((T, J))
        chosen_idx = np.zeros(T, dtype=int)

        X_jt[0] = rng.normal(loc=x_bar[0], scale=sigma_x, size=J)
        a = np.zeros(T)

        a[0] = 0

```

```

epsilon_0 = rng.gumbel(0,1,size=J)
u0 = beta*X_jt[0] - gamma*0 + a[0] + epsilon_0
V[0] = u0
chosen_idx[0] = np.argmax(u0)
x_chosen[0] = X_jt[0, chosen_idx[0]]
Sigma[0] = x_chosen[0]

for t in range(1, T):
    u = np.zeros(J)
    a[t] = 0.23 # 10% chance of promotion
    x_bar[t] = np.mean(x_chosen[:t])
    X_jt[t] = rng.normal(loc=x_bar[t], scale=sigma_x, size=J)
    for j in range(J):
        u[j] = beta*X_jt[t,j] - gamma*Sigma[t-1]**2 + a[t] + epsilon_ijt[j]
    V[t] = u
    chosen_idx[t] = np.argmax(u)
    x_chosen[t] = X_jt[t, chosen_idx[t]]
    Sigma[t] = x_chosen[t] - np.mean(x_chosen[:t+1])

ll = -np.sum(V[np.arange(T), chosen_idx]-logsumexp(V,axis=1,keepdims=True))
prob = np.exp(V) / np.exp(V).sum(axis=1, keepdims=True)
prob_all[s] = prob
x_chosen_all[s] = x_chosen
x_bar_all[s] = x_bar
X_jt_all[s] = X_jt
V_all[s] = V
ll_all[s] = ll
return (x_chosen_all.mean(axis=0), # shape (T,)
        x_bar_all.mean(axis=0),    # shape (T,)
        X_jt_all.mean(axis=0),     # shape (T, J)
        V_all.mean(axis=0),        # shape (T, J)
        prob_all.mean(axis=0),     # shape (T,J)
        ll_all.mean())             # scalar

# Consumer Choice by Regime

colors = plt.cm.tab10(np.linspace(0,1,len(regimes)))

for j in range(J):
    fig, ax = plt.subplots(figsize=(8,4))

    for (beta, gamma, label,ls), color in zip(regimes, colors):
        x_chosen, x_bar, X_jt, V, prob, ll = simulate_cons(
            beta, gamma, sigma_gamma[1], T, J, X_bar, sigma_x, rng
        )

        prob_smooth = np.convolve(prob[:,j], np.ones(5)/5, mode='same')
        ax.plot(np.arange(T), prob_smooth, label=label, color=color, linewidth=1.5,

```



```

        linestyle=ls)
ax.set_title(f"Product_{j+1}", fontsize=9)
ax.set_xlabel("Period")
ax.set_ylim(0.15,0.27)
ax.set_xlim(5, 95)
ax.set_ylabel("Choice_Probability")
ax.legend(fontsize=7, loc="upper_left", bbox_to_anchor=(0,1))
fig.suptitle("Choice_Probability_by_Product_and_Regime", fontsize=13, y=1.01)
plt.tight_layout()
plt.savefig(f"choice_prob_product_{j+1}.pdf", bbox_inches='tight', format='pdf')
plt.show()

def simulate_cons_directed_ads(beta, gamma, sigma_gamma, T, J, X_bar, sigma_x, rng, S
=1000):
    x_chosen_all = np.zeros((S, T))
    x_bar_all = np.zeros((S, T))
    X_jt_all = np.zeros((S, T, J))
    V_all = np.zeros((S, T, J))
    ll_all = np.zeros(S)
    probab_all = np.zeros((S, T, J))

    for s in range(S):

        epsilon_ijt = rng.gumbel(0,1, size=J)

        x_chosen = np.zeros(T) # empty vector of choices per period
        X_jt = np.zeros((T,J))
        x_bar = np.zeros(T)
        x_bar[0] = X_bar

        Sigma = np.zeros(T) # empty vector for the sum of characteristics

        V = np.zeros((T, J))
        chosen_idx = np.zeros(T, dtype=int)

        X_jt[0] = rng.normal(loc=x_bar[0], scale=sigma_x, size=J)
        a = np.zeros(T)

        a[0] = 0
        epsilon_0 = rng.gumbel(0,1,size=J)
        u0 = beta*X_jt[0] - gamma*a[0] + a[0] + epsilon_0
        V[0] = u0
        chosen_idx[0] = np.argmax(u0)
        x_chosen[0] = X_jt[0, chosen_idx[0]]
        Sigma[0] = x_chosen[0]

        for t in range(1, T):
            u = np.zeros(J)

```

```

        ad = 1.0 if j == 1 else 0.0
        x_bar[t] = np.mean(x_chosen[:t])
        X_jt[t] = rng.normal(loc=x_bar[t], scale=sigma_x, size=J)
        for j in range(J):
            u[j] = beta*X_jt[t,j] - gamma*Sigma[t-1]**2 + ad + epsilon_ijt[j]
        V[t] = u
        chosen_idx[t] = np.argmax(u)
        x_chosen[t] = X_jt[t, chosen_idx[t]]
        Sigma[t] = x_chosen[t] - np.mean(x_chosen[:t+1])

    ll = -np.sum(V[np.arange(T), chosen_idx]-logsumexp(V,axis=1,keepdims=True))
    prob = np.exp(V) / np.exp(V).sum(axis=1, keepdims=True)
    prob_all[s] = prob
    x_chosen_all[s] = x_chosen
    x_bar_all[s] = x_bar
    X_jt_all[s] = X_jt
    V_all[s] = V
    ll_all[s] = ll
    return (x_chosen_all.mean(axis=0), # shape (T,)
            x_bar_all.mean(axis=0),    # shape (T,)
            X_jt_all.mean(axis=0),     # shape (T, J)
            V_all.mean(axis=0),        # shape (T, J)
            prob_all.mean(axis=0),     # shape (T,J)
            ll_all.mean())             # scalar

colors = plt.cm.tab10(np.linspace(0,1,len(regimes)))

for j in range(J):
    fig, ax = plt.subplots(figsize=(8,4))

    for (beta, gamma, label,ls), color in zip(regimes, colors):
        x_chosen, x_bar, X_jt, V, prob, ll = simulate_cons(
            beta, gamma, sigma_gamma[1], T, J, X_bar, sigma_x, rng
        )

        prob_smooth = np.convolve(prob[:,j], np.ones(5)/5, mode='same')
        ax.plot(np.arange(T), prob_smooth, label=label, color=color, linewidth=1.5,
                linestyle=ls)
    ax.set_title(f"Product_{j+1}", fontsize=9)
    ax.set_xlabel("Period")
    ax.set_ylim(0.15,0.27)
    ax.set_xlim(5, 95)
    ax.set_ylabel("Choice_Probability")
    ax.legend(fontsize=7, loc="upper_left", bbox_to_anchor=(0,1))
    fig.suptitle("Choice_Probability_by_Product_and_Regime", fontsize=13, y=1.01)
    plt.tight_layout()
    plt.savefig(f"choice_prob_product_{j+1}_directed_ads.pdf", bbox_inches='tight',
                format='pdf')

```

```

plt.show()
#=====#
# Firm Problem and Equilibrium Analysis #
#=====#

def inferred_choice_prob(a, X_j, Sigma_t, lambda_t, beta):
    U_firm = beta*X_j + a - lambda_t*Sigma_t
    U_outside=0
    return np.exp(U_firm) / (1 + np.exp(U_firm))

def solve_firm(Sigma, X_bar, beta, p, c, kappa_vec, a_grid, beta_disc, tol=1e-6):
    T = len(Sigma)
    V = np.zeros((T, len(a_grid)))
    optimal_a = np.zeros(T)

    for i in range(T-1, -1, -1):
        lambda_t = 1 / (Sigma[i]**2) if Sigma[i] > 0 else 1.0
        for j, a in enumerate(a_grid):
            prob = inferred_choice_prob(a, X_bar, Sigma[i], lambda_t, beta)
            pi = prob*((1+a)*p - c - (kappa_vec[i]*a)**2)
            if i < T-1:
                exp_V = np.max(V[i+1])
                V[i,j] = pi + beta_disc*lambda_t*exp_V
            else:
                V[i,j] = pi
        optimal_a = a_grid[np.argmax(V, axis=1)]
    return optimal_a, V

x_chosen, x_bar, X_jt, V_cons, ll = simulate_cons(
    0.5, 0.5, sigma_gamma[1], T, J, X_bar, sigma_x, rng
)
Sigma_path = np.array([np.mean(x_chosen[:t+1]) for t in range(T)])

regimes_1 = regimes[:4]
regimes_2 = regimes[4:]

for fig_num, regime_subset in enumerate([regimes_1, regimes_2], start=1):
    fig, axes = plt.subplots(1, len(regime_subset),
                             figsize=(3*len(regime_subset), 4), sharey=True)

    for ax, (beta, gamma, label) in zip(axes, regime_subset):
        x_chosen, x_bar, X_jt, V_cons, ll = simulate_cons(
            beta, gamma, sigma_gamma[1], T, J, X_bar, sigma_x, rng
        )
        Sigma_path = np.array([np.mean(x_chosen[:t+1]) for t in range(T)])
        kappa_vec = np.clip(
            rng.normal(loc=1/Sigma_path, scale=sigma_lambda[1], size=T),
            0.01, None

```

```

)

optimal_a, _ = solve_firm(
    Sigma_path, X_bar, beta=beta, p=1.0, c=0.5,
    kappa_vec=kappa_vec, a_grid=np.linspace(0, 1, 50), beta_disc=0.9
)

optimal_a_smooth = np.convolve(optimal_a, np.ones(5)/5, mode='same')
ax.plot(np.arange(T), optimal_a_smooth, color='steelblue', linewidth=1.5)
ax.axhline(np.mean(optimal_a_smooth), color='black', linestyle='--',
           linewidth=1, label=f"Mean={np.mean(optimal_a_smooth):.2f}")
ax.set_title(label, fontsize=8)
ax.set_xlabel("Period")
ax.legend(fontsize=8)

axes[0].set_ylabel("Optimal_$$$")
fig.suptitle("Firm's_Optimal_Advertising_by_Consumer_Regime", fontsize=13, y=1.01)
plt.tight_layout()
plt.savefig(f"firm_optimal_a_by_regime_{fig_num}.pdf",
           dpi=300, bbox_inches='tight', format='pdf')
plt.show()

```